

在 Cloud Run 建構及部署採用 MCP 伺服器的 ADK 代理

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/use-mcp-server-on-cloud-run-with-an-adk-agent?hl=zh-tw>

步驟數：9

MCP 伺服器提供工具後，現在要建構代理的「大腦」。在本實驗室中，您將使用 Agent Development Kit (ADK) 建立智慧型代理，讓代理瞭解目標，並使用實驗室 1 的工具達成目標。

目錄

1. [1. 簡介](#)
2. [2. 部署至 Cloud Run 的理由](#)
3. [3. 事前準備](#)
4. [4. 設定環境](#)
5. [5. 建立代理程式工作流程](#)
6. [6. 使用 ADK CLI 部署代理](#)
7. [7. 測試已部署的代理](#)
8. [8. 清理環境](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

本實驗室將著重於實作及部署用戶端代理服務。您將使用 [Agent Development Kit \(ADK\)](#) 建構 AI 代理，並串連遠端工具，例如在 [實驗室 1](#) 中建立的 MCP 伺服器。本實驗室展示「關注點分離」這項關鍵架構原則，也就是讓獨立的推論層 (代理) 和工具層 (MCP 伺服器)，透過安全的 API 互相通訊。

在實驗室 1 中，您建立了 MCP 伺服器，為大型語言模型 (可透過 Gemini CLI 互動) 提供虛構動物園的動物資料。在本實驗室中，我們將為虛構動物園建構導覽員代理。這個代理會透過實驗室 1 的 MCP 伺服器存取園內動物的詳細資料，並運用維基百科打造最佳導覽體驗。



最後，我們會將導覽員代理部署至 Google Cloud Run，讓所有動物園遊客都能使用，而不只是在本機執行。

必要條件

- 運行在 Cloud Run 的 MCP 伺服器，或與伺服器相關聯的服務網址。
- 已啟用計費功能的 Google Cloud 專案。

課程內容

- 如何建構 Python 專案，以利部署 ADK。
- 如何使用 google-adk 導入會使用工具的代理。
- 如何將代理連線至遠端 MCP 伺服器，以取用工具組。
- 如何將 Python 應用程式以無伺服器容器的形式部署至 Cloud Run。
- 如何使用 IAM 角色設定安全的服務對服務驗證。
- 如何刪除 Cloud 資源，避免產生後續費用。

軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
 - 網路瀏覽器，例如 [Chrome](#)
-

2. 部署至 Cloud Run 的理由

Cloud Run 是無伺服器平台，非常適合用來託管 ADK 代理。您只需要專心編寫程式碼，不必管理基礎架構，相關作業交由我們代勞即可。

這個平台好比快閃店，只有在顧客 (要求) 到來時才需要營業及使用資源；沒有顧客時，商店會完全關閉，您不需為閒置店面付費。

主要功能

可隨處執行容器：

- 您只要準備內含應用程式的容器 (Docker 映像檔)。
- Cloud Run 會在 Google 基礎架構上執行容器。
- 不必修補作業系統、設定虛擬機器或煩惱資源調度事宜。

自動調整資源配置：

- 當應用程式無人使用時，所有執行個體都不會運行 (縮減至零個執行個體，可節省成本)。
- 如果收到 1,000 個要求，系統會視需要啟動足量的執行個體。

預設為無狀態：

- 每項要求都能傳送至不同的執行個體。
- 如需儲存狀態，請使用 Cloud SQL、Firestore 或 Memorystore 等獨立於應用程式的服務。

支援任何語言或架構：

- 無論應用程式是以 Python、Go、Node.js、Java 或 .Net 編寫，只要能在 Linux 容器中執行，Cloud Run 就都支援。

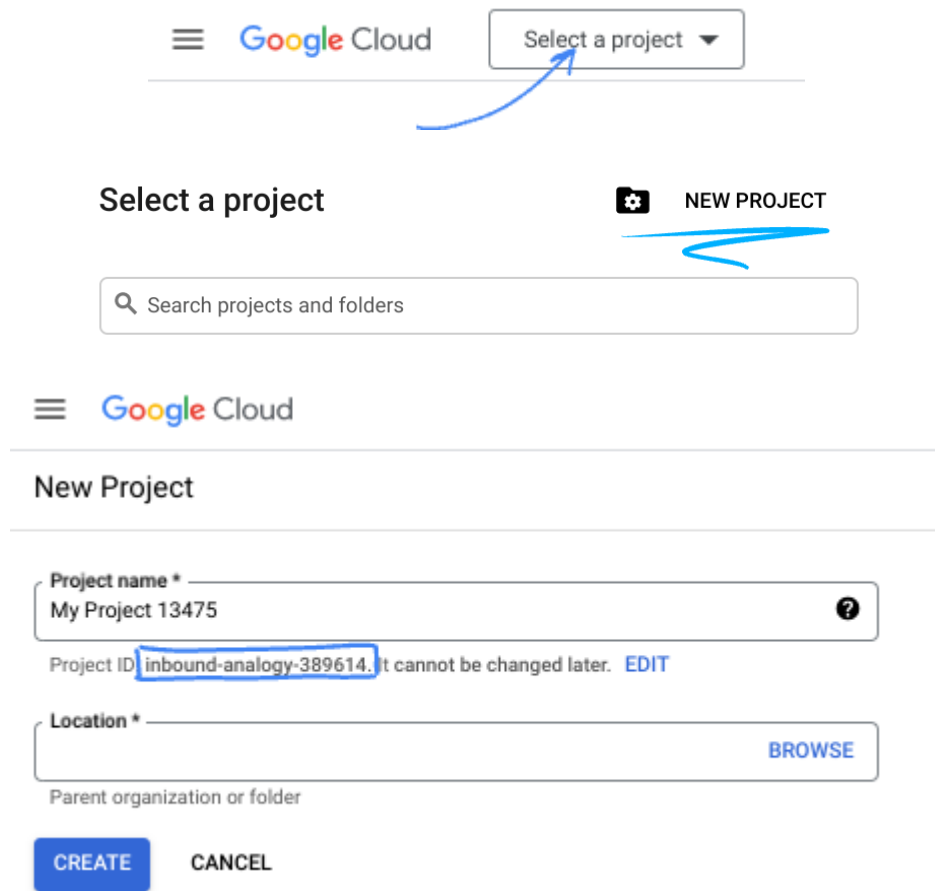
用多少付多少：

- [以要求為依據計費](#)：按要求次數 + 運算時間 (可精確至 100 毫秒) 計費。
 - [以執行個體為依據的計費模式](#)：針對整個執行個體生命週期計費 (不收取要求費用)。
-

3. 事前準備

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the value 'My Project 13475', a 'Project ID' field with the value 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. There are also buttons for 'CREATE' and 'CANCEL' at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

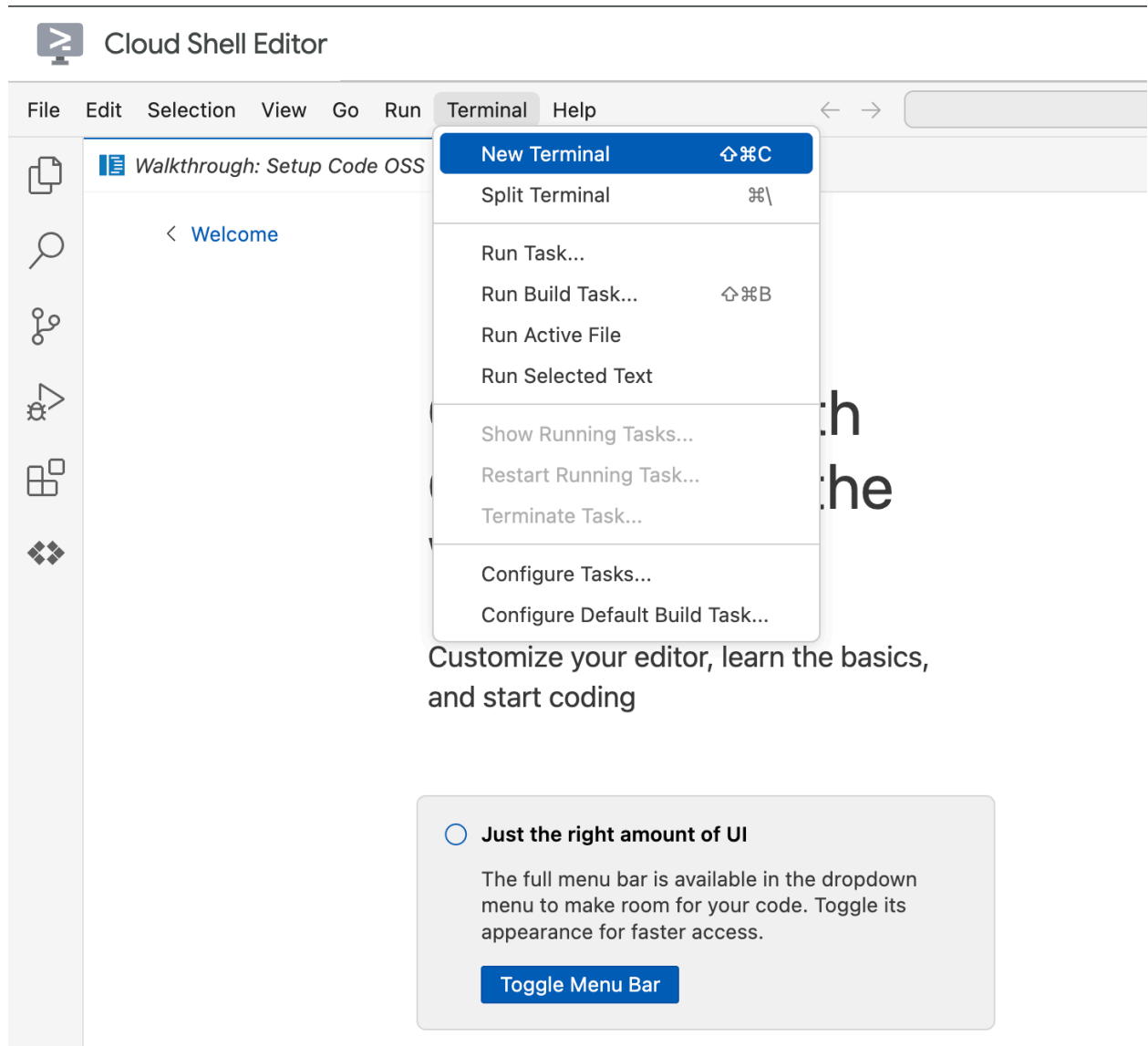
注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用

者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

1. 前往 [Cloud Shell 編輯器](#)
2. 如果畫面底部未顯示終端機，請開啟終端機：
 - 按一下「終端機」。
 - 按一下「New Terminal」(新增終端機)



3. 在終端機中，使用下列指令設定專案。如果您已完成實驗室 1，請確認您使用的是相同的專案 ID：

```
gcloud config set project [YOUR-PROJECT-ID]
```

如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```

4. 如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/n)?` 提示，可能是專案 ID 輸入有誤。按下 `n` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 4 · 約 15 分鐘

4. 設定環境

啟用 API 並設定環境變數

啟用所有必要服務：

```
gcloud services enable \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com
```

預期的輸出內容：

```
Operation "operations/acat.p2-[GUID]" finished successfully.
```

建立專案目錄。

這個指令會建立實驗室的主要資料夾，用來存放代理的原始碼。

```
cd && mkdir zoo_guide_agent && cd zoo_guide_agent
```

建立 `requirements.txt` 檔案。這個檔案會列出代理程式所需的 Python 程式庫。下列指令會建立並填入檔案。

```
cloudshell edit requirements.txt
```

```
google-adk==1.14.0  
langchain-community==0.3.27  
wikipedia==1.4.0
```

為目前的專案 ID 和專案編號設定變數，並為專案建立專屬的[服務帳戶](#)。這是執行這些指令的更健全方式。

```
export PROJECT_ID=$(gcloud config get-value project)  
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")  
export SA_NAME=lab2-cr-service  
export SERVICE_ACCOUNT="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"
```

```
gcloud iam service-accounts create ${SA_NAME} \  
  --display-name="Service Account for lab 2 "
```

建立並開啟 `.env` 檔案，以驗證 `zoo_guide_agent` 目錄中的代理。

```
cloudshell edit .env
```


cloudshell edit 指令會在終端機上方的編輯器中開啟 `.env` 檔案。在 `.env` 檔案中輸入下列內容，然後返回終端機。

```
MODEL="gemini-2.5-flash"
```

新增 **MCP 伺服器網址**。如果您已完成實驗室 1，請按照下列步驟使用在實驗室 1 中建立的 MCP 伺服器：

附註：如果您已完成實驗室 1，並想使用該實驗室建立的 MCP 伺服器，請執行下列三項指令。如果您已在直播活動中使用系統提供的公開 MCP 伺服器連結，則不需要執行這兩項指令。

授予代理程式服務帳戶權限，以便在 Vertex AI 上使用 Gemini，並呼叫遠端 MCP 伺服器：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/aiplatform.user"
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/run.invoker"
```

將實驗室 1 的 MCP 伺服器網址儲存至環境變數。

```
echo -e "\nMCP_SERVER_URL=https://zoo-mcp-server-{$PROJECT_NUMBER}.us-west1.run.app/mcp" >>
.env
```

如果您使用公開 **MCP 伺服器連結**，請執行下列指令，並將 `PROJECT_NUMBER` 替換為系統提供的內容。

```
echo -e "\nMCP_SERVER_URL=https://zoo-mcp-server-{$PROJECT_NUMBER}.us-west1.run.app/mcp" >>
.env
```

步驟 5 · 約 20 分鐘

5. 建立代理程式工作流程

建立 `init.py` 檔案

建立 `init.py` 檔案。用來告訴 Python `zoo_guide_agent` 目錄是套件。

```
cloudshell edit __init__.py
```

先前的指令會開啟程式碼編輯器。在 `__init__.py` 加入以下程式碼：

```
from . import agent
```

建立主要的 `agent.py` 檔案

建立主要的 `agent.py` 檔案。這個指令會建立 Python 檔案，並貼上多代理系統的完整程式碼。

```
cloudshell edit agent.py
```

匯入和初始設定

下方程式碼區塊的第一段，會從 ADK 和 Google Cloud 匯入所有必要程式庫、設定記錄功能，以及從 `.env` 檔案載入環境變數，這些是存取模型和伺服器網址的關鍵。

將下列程式碼新增至 `agent.py` 檔案：

```
import os
import logging
import google.cloud.logging
from dotenv import load_dotenv

from google.adk import Agent
from google.adk.agents import SequentialAgent
from google.adk.tools.mcp_tool.mcp_toolset import MCPToolset, StreamableHTTPConnectionParams
from google.adk.tools.tool_context import ToolContext
from google.adk.tools.langchain_tool import LangchainTool

from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

import google.auth
import google.auth.transport.requests
import google.oauth2.id_token

# --- Setup Logging and Environment ---

cloud_logging_client = google.cloud.logging.Client()
cloud_logging_client.setup_logging()

load_dotenv()

model_name = os.getenv("MODEL")
```

定義工具 (代理的功能)



代理的實用度取決於所使用的工具。在本節中，我們將定義代理的所有功能，包括儲存資料的自訂函式、連線至安全 MCP 伺服器的 MCP 工具，以及維基百科工具。

將下列程式碼新增至 `agent.py` 底部：

```

# Greet user and save their prompt

def add_prompt_to_state(
    tool_context: ToolContext, prompt: str
) -> dict[str, str]:
    """Saves the user's initial prompt to the state."""
    tool_context.state["PROMPT"] = prompt
    logging.info(f"[State updated] Added to PROMPT: {prompt}")
    return {"status": "success"}


# Configuring the MCP Tool to connect to the Zoo MCP server

mcp_server_url = os.getenv("MCP_SERVER_URL")
if not mcp_server_url:
    raise ValueError("The environment variable MCP_SERVER_URL is not set.")


def get_id_token():
    """Get an ID token to authenticate with the MCP server."""
    target_url = os.getenv("MCP_SERVER_URL")
    audience = target_url.split('/mcp/')[0]
    request = google.auth.transport.requests.Request()
    id_token = google.oauth2.id_token.fetch_id_token(request, audience)
    return id_token


"""
# Use this code if you are using the public MCP Server and comment out the code below
defining mcp_tools
mcp_tools = MCPToolset(
    connection_params=StreamableHTTPConnectionParams(
        url=mcp_server_url
    )
)
"""

mcp_tools = MCPToolset(
    connection_params=StreamableHTTPConnectionParams(
        url=mcp_server_url,
        headers={
            "Authorization": f"Bearer {get_id_token()}",
        },
    ),
)


# Configuring the Wikipedia Tool
wikipedia_tool = LangchainTool(
    tool=WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
)

```

三種工具說明

1. add_prompt_to_state

這項工具會記住動物園遊客提出的問題。當遊客詢問「獅子在哪一區？」時，此工具會將問題儲存到代理的記憶體中，這樣工作流程中的其他代理就知道要研究什麼問題。

運作機制：Python 函式會將訪客的提示詞寫入共用的 `tool_context.state` 字典。這個工具情境可以視為代理在單一對話中的短期記憶。工作流程中的下一個代理，可以讀取前一個代理儲存至狀態的資料。

2. MCPToolset 🦁

這項設定用於將導覽員代理程式連線至實驗室 1 中建立的動物園 MCP 伺服器。這個伺服器具備特殊工具，可查詢園內動物的特定資訊，例如名稱、年齡和圈養園區。

運作機制：此工具會以安全的方式連線至動物園的私人伺服器網址，並透過 `get_id_token` 自動取得安全「金鑰卡」(服務帳戶 ID 權杖)，以證明自己的身分並取得存取權。

3. LangchainTool 🌐

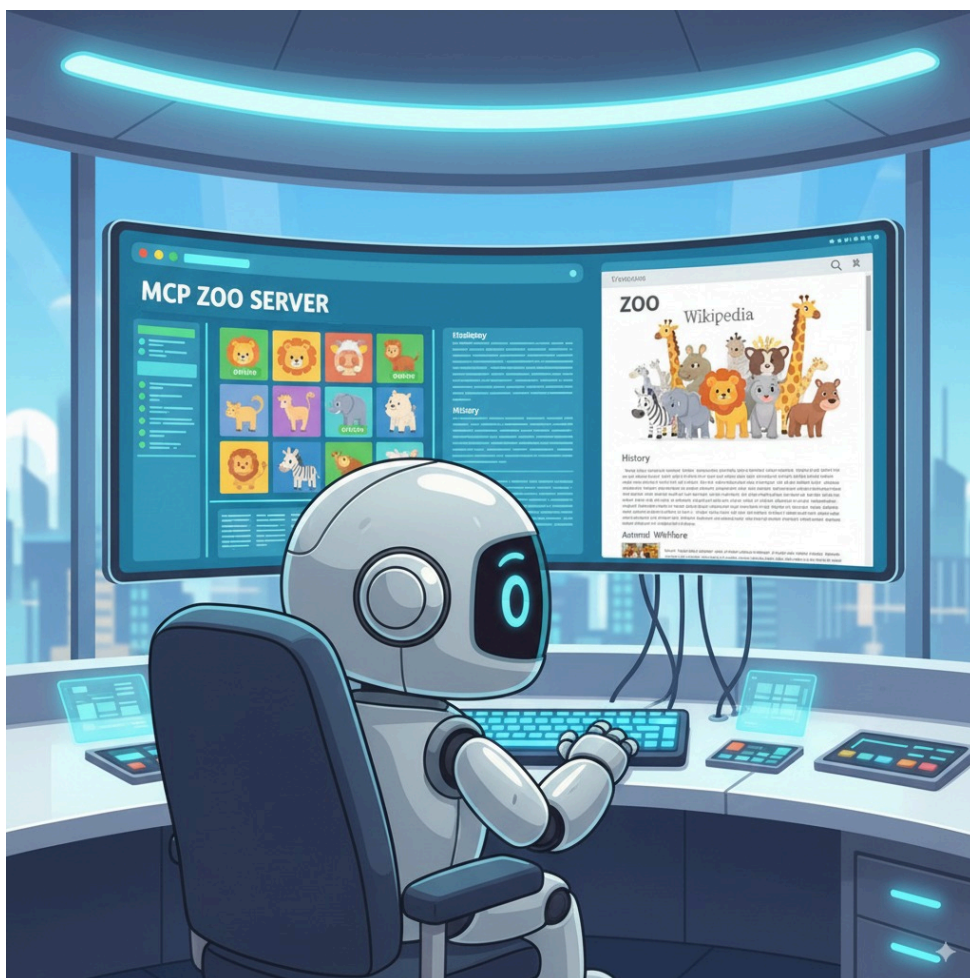
這個導覽員代理具備一般世界知識。當遊客提出動物園資料庫未收錄的問題時，例如「獅子在野外吃什麼？」，這項工具會指示代理前往維基百科搜尋答案。

運作機制：這項工具擔任中介轉換的角色，讓代理使用 LangChain 程式庫中預先建構的 `WikipediaQueryRun` 工具。

資源：

- [MCP 工具組](#)
- [函式工具](#)
- [狀態](#)

定義專用代理



接下來，我們要定義研究人員代理和回覆格式化工具代理。研究人員代理是這項作業的「大腦」，這個代理會從共用的 `State` 取得使用者的提示詞、綜覽可運用的強大工具 (動物園的 MCP 伺服器工具和維基百科工具)，然後決定要使用哪些工具來搜尋答案。

回覆生成代理的角色是呈現資訊。這個代理不會使用任何工具尋找新資訊，而是會擷取「研究人員」代理收集的原始資料 (透過 `State` 傳遞)，並運用大型語言模型的語言組織能力，將資料轉換成親切的對話式回覆。

將下列程式碼新增至 `agent.py` 底部：

```

# 1. Researcher Agent
comprehensive_researcher = Agent(
    name="comprehensive_researcher",
    model=model_name,
    description="The primary researcher that can access both internal zoo data and external
knowledge from Wikipedia.",
    instruction="""
You are a helpful research assistant. Your goal is to fully answer the user's PROMPT.
You have access to two tools:
1. A tool for getting specific data about animals AT OUR ZOO (names, ages, locations).
2. A tool for searching Wikipedia for general knowledge (facts, lifespan, diet, habitat).

First, analyze the user's PROMPT.
- If the prompt can be answered by only one tool, use that tool.
- If the prompt is complex and requires information from both the zoo's database AND
Wikipedia,
    you MUST use both tools to gather all necessary information.
- Synthesize the results from the tool(s) you use into preliminary data outputs.

PROMPT:
{{ PROMPT }}
""",
    tools=[
        mcp_tools,
        wikipedia_tool
    ],
    output_key="RESEARCH_DATA" # A key to store the combined findings
)

# 2. Response Formatter Agent
response_formatter = Agent(
    name="response_formatter",
    model=model_name,
    description="Synthesizes all information into a friendly, readable response.",
    instruction="""
You are the friendly voice of the Zoo Tour Guide. Your task is to take the
RESEARCH_DATA and present it to the user in a complete and helpful answer.

- First, present the specific information from the zoo (like names, ages, and where to
find them).
- Then, add the interesting general facts from the research.
- If some information is missing, just present the information you have.
- Be conversational and engaging.

RESEARCH_DATA:
{{ RESEARCH_DATA }}
""")

```


工作流程代理

工作流程代理好比動物園導覽員的「後勤」經理，負責接收研究要求，確保我們在上方定義的兩個代理按正確順序工作：先研究，再生成回覆。這樣一來，回答訪客的問題時，就有一套可預測且可靠的程序能遵循。

運作機制：工作流程代理會使用 `SequentialAgent`，這是一種不會自行思考的特殊代理。他只負責按照固定順序執行 `sub_agents` 清單中的子代理 (研究人員和回覆生成)，自動將共用記憶體從一個子代理傳遞給下一個。

將這個程式碼區塊新增至 `agent.py` 底部：

```
tour_guide_workflow = SequentialAgent(  
    name="tour_guide_workflow",  
    description="The main workflow for handling a user's request about an animal.",  
    sub_agents=[  
        comprehensive_researcher, # Step 1: Gather all data  
        response_formatter,        # Step 2: Format the final response  
    ]  
)
```

組裝主要工作流程



這個代理程式會指定為 `root_agent`，ADK 架構會將這個代理程式做為所有新對話的起點。主要任務是協調所有程序，並擔任初始控管者，管理對話的第一輪。

請將最後一段程式碼區塊新增至 `agent.py` 底部：

```
root_agent = Agent(  
    name="greeter",  
    model=model_name,  
    description="The main entry point for the Zoo Tour Guide.",  
    instruction="""  
    - Let the user know you will help them learn about the animals we have in the zoo.  
    - When the user responds, use the 'add_prompt_to_state' tool to save their response.  
    After using the tool, transfer control to the 'tour_guide_workflow' agent.  
    """,  
    tools=[add_prompt_to_state],  
    sub_agents=[tour_guide_workflow]  
)
```

您的 `agent.py` 檔案現已建構完成！透過這種方式建構代理，您將能瞭解每個元件（工具、作業代理和管理代理）在打造最終智慧型系統時所扮演的角色。接著是部署時間！

6. 使用 ADK CLI 部署代理

本機程式碼和 Google Cloud 雲端專案就緒後，就可以部署代理了。您會使用 `adk deploy cloud_run` 指令，輕鬆將整個部署工作流程自動化。這個單一指令會封裝程式碼、建構容器映像檔、將映像檔推送至 Artifact Registry，並在 Cloud Run 上啟動服務，讓使用者透過網路存取。

部署

執行下列指令來部署代理程式。`uvx` 指令可讓您執行以 Python 套件形式發布的指令列工具，而不需在全域安裝這些工具。

注意：這個指令需要幾分鐘才能執行完畢。

```
# Run the deployment command
uvx --from google-adk==1.14.0 \
adk deploy cloud_run \
  --project=$PROJECT_ID \
  --region=us-west1 \
  --service_name=zoo-tour-guide \
  --with_ui \
  . \
  -- \
  --labels=dev-tutorial=codelab-adk \
  --service-account=$SERVICE_ACCOUNT
```

接受提示

系統可能會顯示下列提示：

```
Deploying from source requires an Artifact Registry Docker repository to store built
containers. A repository named [cloud-run-source-deploy] in region
[us-west1] will be created.
```

```
Do you want to continue (Y/n)?
```

輸入 Y 並按下 ENTER 鍵。

系統可能會顯示下列提示：

```
Allow unauthenticated invocations to [your-service-name] (y/N)?.
```

在本實驗室中，我們希望允許未經驗證的叫用，方便進行測試，因此請輸入 `y` 並按下 Enter 鍵。

注意：任何知道網址的人都能存取這個代理，因此最適合用於測試。

取得部署作業連結

執行成功後，指令會提供已部署 Cloud Run 服務的網址。(看起來會像 `https://zoo-tour-guide-123456789.us-west1.run.app`)。複製這個網址，以供下一個工作使用。

7. 測試已部署的代理

現在代理已在 Cloud Run 運作，請執行測試來確認是否部署成功，並驗證代理能否正常運作。您將使用公開服務網址 (類似 `https://zoo-tour-guide-123456789.us-west1.run.app/`) 存取 ADK 的網頁介面，並與代理互動。

在網路瀏覽器中開啟公開 Cloud Run 服務網址。由於您使用了 `--with_ui flag`，因此應該會看到 ADK 開發人員 UI。

將右上角的 `Token Streaming` 切換為開啟。

現在您可以與動物園導覽員代理互動了。

輸入 `hello` 並按下 Enter 鍵，即可開始新對話。

請查看結果，代理應會迅速回覆問候語：

"Hello! I'm your Zoo Tour Guide. I can help you learn about the amazing animals we have here. What would you like to know or explore today?"

向虛擬服務專員提出以下問題：

Where can I find the polar bears in the zoo and what is their diet?

Hi

Welcome to the zoo! I can help you learn about the animals we have here. What would you like to know?

Where can I find the polar bears in the zoo and what is their diet?

⚡ add_prompt_to_state

✓ add_prompt_to_state

⚡ transfer_to_agent

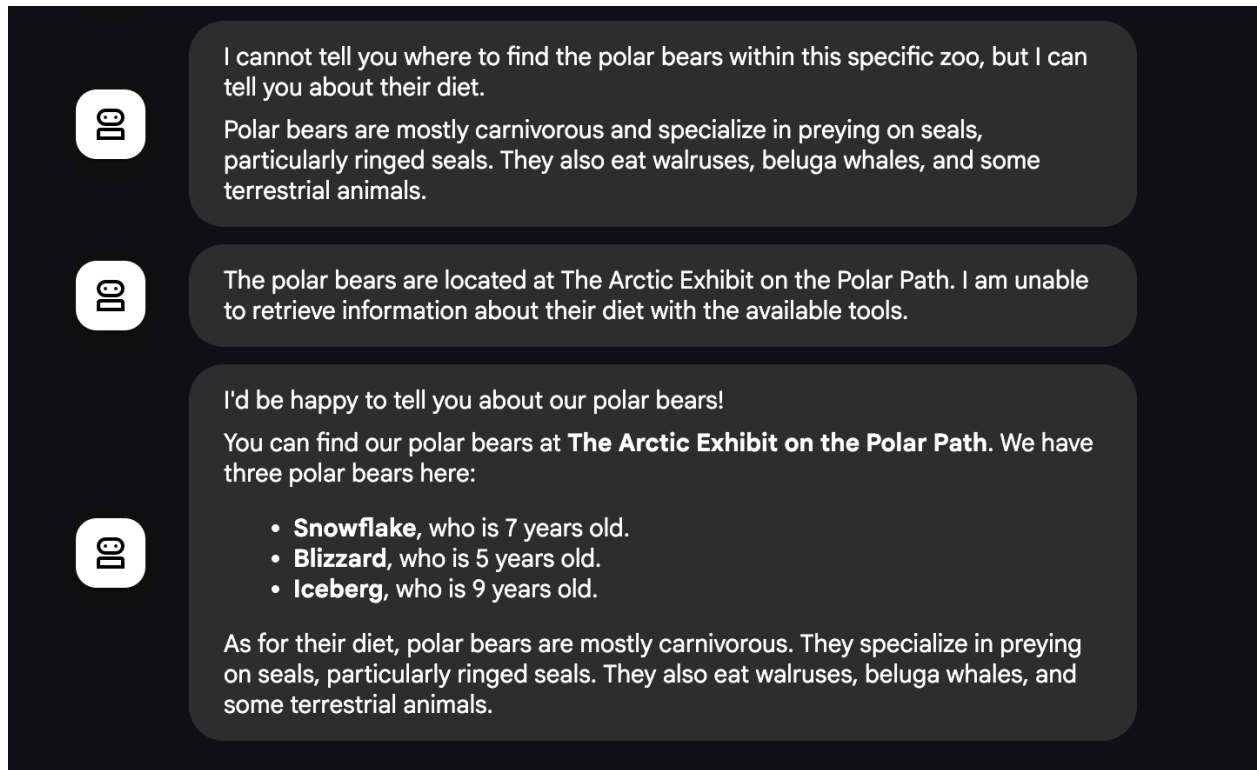
✓ transfer_to_agent

⚡ wikipedia

⚡ get_animals_by_species

✓ wikipedia

✓ get_animals_by_species



代理流程說明

您的系統就像一個智慧型多代理團隊，會按明確程序執行作業，確保使用者從提出問題到獲得最終詳細解答的過程順暢有效率。

1. 動物園迎賓員 (迎賓櫃台)

整個流程從迎賓代理開始。

用途：展開對話。這個代理會問候使用者，並詢問他們想瞭解哪種動物。

使用的工具：當使用者回覆時，迎賓代理會使用 `add_prompt_to_state` 工具完整擷取輸入的文字 (例如「介紹一下獅子」)，並儲存在系統記憶體中。

轉接：儲存提示詞後，迎賓代理會立即將控制權轉給子代理 `tour_guide_workflow`。

2. 全方位研究人員 (超級研究人員)

這是主要工作流程的第一步，也是整個作業的「大腦」。您不必仰賴龐大的團隊，只要一個技能高超的代理就能存取所有可用資訊。

用途：分析使用者的問題，並聰明擬定計畫。這個代理會運用強大的語言模型，判斷是否需要：

- 取用動物園記錄的內部資料 (透過 MCP 伺服器)。
- 從網路 (透過 Wikipedia API) 搜尋一般知識。
- 同時運用上述兩個管道，以回答複雜問題。

執行動作：執行必要工具，收集所有需要的原始資料。舉例來說，如果遊客問「園內的獅子幾歲？牠們在野外吃什麼？」，模型會呼叫 MCP 伺服器取得年齡資訊，並呼叫維基百科工具取得飲食資訊。

3. 回覆生成 (資訊呈現者)

全方位研究人員收集所有事實後，就會交由最後的代理來呈現。

用途：擔任友善的動物園導覽員，將原始資料 (可能來自一個或兩個來源) 整理成實用回覆。

執行動作：將所有資訊統整成單一、連貫且實用的答案。這個代理會按照指示，先提供該動物園的具體資訊，再補充有趣的一般知識。

最終結果：代理會生成完整詳細的文字答案，並顯示在對話視窗中。

請參閱下列資源，進一步瞭解如何建構代理：

1. [ADK 說明文件](#)
 2. [為 ADK 代理建構自訂工具](#)
-

步驟 8 · 約 10 分鐘

8. 清理環境

```
gcloud run services delete zoo-tour-guide --region=us-west1 --quiet  
gcloud artifacts repositories delete cloud-run-source-deploy --location=us-west1 --quiet
```

步驟 9 · 約 1 分鐘

9. 恭喜

恭喜您完成本程式碼研究室。

涵蓋內容

- 如何建構 Python 專案，以利使用 ADK 指令列介面部署。
- 如何使用 SequentialAgent 和 ParallelAgent 實作多代理工作流程。
- 如何使用 MCPToolset 連線至遠端 MCP 伺服器，以取用其中的工具。
- 如何整合 Wikipedia API 等外部工具，擴充內部資料。
- 如何使用 `adk deploy` 指令，將代理程式以無伺服器容器的形式部署至 Cloud Run。

問卷調查
